Functional Programming | FP

Summary

CONTENTS

1. Functional Language 2
   1.1. Terms 2
   1.2. Basic features of Functional Programming 2
   1.3. Programming Paradigms 2
   1.4. Algebra 2
2. Haskell 3
   2.1. Function definition 3
      2.1.1. Guarded Equations 3
      2.1.2. Pattern matching 3
      2.1.3. List patterns 3
      2.1.4. Lambda expressions 3
   2.2. Function application 3
      2.2.1. Point-free notation 3
   2.3. Operators 4
   2.4. Conditional expressions 4
   2.5. List comprehension 4
   2.6. Types 4
      2.6.1. Type declarations 4
      2.6.2. Data declarations 5
      2.6.3. Polymorphism 7
      2.6.4. Typeclasses 7
      2.6.5. Newtype declarations 7
      2.6.6. Instances 8
   2.7. Modules 11
   2.8. Lazy evaluation 11
   2.9. Equational reasoning 12
   2.10. Side effects 12
   2.11. TODO: 12
3. Lambda Calculus 13

1. FUNCTIONAL LANGUAGE

Term	*Definition*
Functional programming	Style of programming with the focus on application of functions to arguments
Functional language	Languages that encourage functional programming

1.1. TERMS

Term	*Definition*
Mutual recursion	Two or more functions defined recursively in terms of each other (=calling each other)
Higher-order functions	A function is called higher-order if it takes a function as an argument or returns a function as a result.
Effectful programming	“Effectual” simply means that arguments and return values are no longer just plain (pure) values, but may also have so-called “effects” such as: – The possibility of failure, e.g. using the option type Maybe – Aggregating multiple results, e.g. using the list type [] – Performing IO, e.g. using the action type IO

1.2. BASIC FEATURES OF FUNCTIONAL PROGRAMMING

– Referential transparency
– Functions as first-class citizens
– Higher-order functions
– Algebraic data types
– Pattern matching
– Recursion
– Types & type inference
– Haskell Specific: Type classes, Functors, Applicatives, Monads
– A Declarative Programming Paradigm.
– Foundation: Church’s Lambda calculus.
– Pure: No (or controlled) mutable state. Expressions are (by default) side effect free.

1.3. PROGRAMMING PARADIGMS

TODO:

1.4. ALGEBRA

TODO:

– Equational reasoning
– Proving correctness of programs

2. HASKELL

2.1. FUNCTION DEFINITION

```
funName :: (TypeBound a) => a -> b          -- type definition
funName x = show x                          -- implementation

complexFun :: (Ord a) => [a] -> [a] -> [a]
complexFun [] _                 = []        -- pattern matching
complexFun _ []                 = []
complexFun (x:xs) (y:ys)        = [x, y]    -- list patterns
complexFun xs ys
                | length xs > 3 = xs        -- guards
                | otherwise     = ys
```

2.1.1. Guarded Equations

```
abs n
    | n >= 0 = n
    | otherwise = -n
```

2.1.2. Pattern matching

Patterns consist of variables and data constructors. They are matched in order, so more specific patterns should come first.

```
not :: Bool -> Bool
not False = True
not True = False
-- ==
not b = case b of
          True -> False
          False -> True
```

2.1.3. List patterns

```
head :: [a] -> a
head (x : _) = x
tail :: [a] -> [a]
tail (_ : xs) = xs
```

2.1.4. Lambda expressions

```
-- \boundvar -> body
(\x -> x + x) 5
```

2.2. FUNCTION APPLICATION

```
sum [1..5]
```

2.2.1. Point-free notation

== Function composition

```
odd x = not (even x)
odd = not . even
```

The term “point-free” does not refer to the “.” character used for function composition (the number of which typically increase), refers to the fact that the definition does not mention the data points on which functions act.

Every definition has a point-free form that can be computed automatically! http://pointfree.io

2.3. OPERATORS

- Prefix notation is used for function application: `max 7 2`
- If infix notation is desired, one may use so-called operators (e.g. +): `7 + 2`
- Functions can be converted into operators using backticks: `7 `div` 2`
- Operators can be converted into functions using brackets: `(+) 7 2`

Notes:

- Operator symbols are formed from one or more symbol characters !#$%&*+./<=>?@^|- : (or any Unicode symbol or punctuation).
- An operator symbol starting with : may only be used for data constructors (e.g. : is the constructor for lists).
- A so-called fixity declaration is used to specify the associativity and precedence level (1 (highest) to 9 (lowest)) of an operator. E.g.:
  - `infixl 6 +` specifies (+) to be left associative with level 6
  - `infixr 5 :` specifies (:) to be right associative with level 5
  - `infix 4 ==` specifies (==) to be neither left nor right associative (making parentheses mandatory while nesting), and with level 4
- Any operator lacking a fixity declaration is assumed to be `infixl 9`.
- Function application has a higher precedence than any infix operator.

2.4. CONDITIONAL EXPRESSIONS

Always ternary.

```
abs :: Int → Int
abs n = if n ≥ 0 then n else -n
```

2.5. LIST COMPREHENSION

```
[x ^ 2 | x ← [1 .. 5]]                  -- [1, 4, 9, 16, 25]
-- guards
[x | x ← [1 .. 12], 12 `mod` x == 0]    -- [2, 3, 4, 6, 12]
-- multiple generators
[(x, y) | x ← [1, 2], y ← [4, 5]]       -- [(1,4),(1,5),(2,4),(2,5)]
-- dependant generators
[(x, y) | x ← [1 .. 2], y ← [x .. 2]] -- [(1,1),(1,2),(2,2)]
```

2.6. TYPES

A Type in Haskell is a name for a collection of related values.

2.6.1. Type declarations

In Haskell, a new name for an existing type can be defined using a type declaration.

```
type String = [Char]
```

Type declarations can also have type parameters

```haskell
type Pair a = (a, a)

mult :: Pair Int → Int
mult (m, n) = m * n
```

Type declarations can be nested but cannot be recursive

```haskell
type Pos = (Int, Int)
type Trans = Pos → Pos      -- OK

type Tree = (Int, [Tree])   -- NOK
```

2.6.2. Data declarations

We can define new custom data types by declaring new types and functions that return values of these types. These functions (a.k.a. “data constructors” or “constructors”) are special since they cannot have a definition that results in a reduction rule. Thery therefore remain unchanged during execution and can be used to hold information.

```haskell
data List a = Nil | Cons a (List a)
```

`List` has values of the form of `Nil` and `Cons` where ***a*** is a generic. `Nil` and `Cons` can be viewed as functions that construct values of type `List`.

A completely new type can be defined by specifying its values using a data declaration.

```haskell
data Bool = False | True
```

- The two values `False` and `True` are called the constructors for the type `Bool`.
- Type and constructor names must always begin with an upper-case letter.
- Data declarations are similar to context free grammars. The former specifies the values of a type, the latter the sentences of a language.

Such data types are often referred to as algebraic datatypes.

- Type variables within types are implicitly universally quantified at the topmost level. For example, `a → Maybe a` actually denotes `forall {a}. a → Maybe a`. The `{}` brackets around `a` denotes that its instantiation can only be done automatically.
- You may ask ghci to explicitly print universal quantifiers as follows:

```
Prelude> :set -fprint-explicit-foralls
Prelude> :t Just
Just :: forall {a}. a → Maybe a
```

- You may explicitly specify the universal quantification in polymorphic type signatures using the `ExplicitForAll` extension.

Maybe and Pair are called Type Constructors since they construct one type from another. This behaviour is expressed using kinds, which is the type of a “type”. The type system for kinds is very simple:

```
K ::= * | K → K
```

`*` Kind of ordinary/proper/concrete/basic types
`→` Kind of “type constructor” or “type operators”

- This is essentially the “simply typed lambda calculus” one level up, with one base type.
- Ordinary/proper/concrete/basic types are nothing more than nullary type constructors/operators. Analogy: A constant is nothing more than a nullary function.
- The word “type” is therefore often used for any type-level expression: ordinary/proper/concrete/basic types, and for type constructors/operators.

Examples: (Only types of kind * can have a value)

Kind	`*`	`*`	`*`	`*`	`*`	`* → *`	`* → * → *`
Type	`Bool`	`Char`	`[a] → [a]`	`Maybe Char`	`a → Maybe a`	`Maybe`	`(,)`
Value	`True`	`'a'`	`reverse`	`Just 'a'`	`Just`		

`(→)` is also a type constructor!

Kind	`*`	`*`	`* → * → *`	`* → *`	`* → *`
Type	`Bool → Bool`	`(→) Bool Bool`	`(→)`	`(→) Bool`	`(→) a`
Value	`not`	`not`			

The operator `(→)` is overloaded:
– In the context of a type (e.g. id::a->a), it denotes the type constructor for function types.
– In the context of a kind (e.g. List::*->*), it denotes the kind constructor for type constructors.

2.6.2.1. Records

Convenient syntax when data constructors have multiple parameters:

```
data Person = Person
  { name :: String
  , age :: Int
  , children :: [Person]
  } deriving (Show)

-- ⟺

data Person = Person String Int [Person] -- positional constructor
  deriving (Show)
-- v selector functions v
name :: Person → String
name (Person n _ _) = n

age :: Person → Int
age (Person _ a _) = a

children :: Person → [Person]
children (Person _ _ c) = c
```

Construction:

```
alice :: Person
alice = Person "Alice" 5 []

bob :: Person
bob = Person {name = "Bob", age = 35, children = [alice]}
```

Derived show contains labels and updates can use field labels:

```
--  >>> alice{age = age alice + 1}
-- Person {name = "Alice", age = 6, children = []}
```

2.6.3. Polymorphism

= “Many forms”

Term	*Definition*
Ad-hoc Polymorphism	Function with the same name denotes different implementations (function overloading, Interfaces)
Parametric Polymorphism	Code written to work with many possible types (OO: generics)
Subtype Polymorphism	One type (subtype) can be substituted for another (supertype)

2.6.4. Typeclasses

Haskell uses type classes to achieve ad-hoc polymorphism.

```
elem :: Eq a ⇒ a → [a] → Bool -- Eq is a typeclass
```

Type classes are declared using class declarations:

```
-- Eq = name of the declared type class
class Eq a where                -- a = type parameter
  (==), (/=) :: a → a → Bool -- Required to be a member of Eq
  x /= y = not (x == y)         -- Default implementation (optional)
```

Type classes may be extended:

```
-- Eq = name of the type class to be extended
class Eq a ⇒ Ord a where       -- Ord = name of resulting type class
  (<), (≤), (≥), (>) :: a → a → Bool -- Additional requirements
```

- All members of the type class `Ord` are also members of the type class `Eq`.
- Members of `Ord` therefore also require `(==)` to be defined.

A type can be declared to be an instance of a type class using an instance declaration:

```
-- In order for (Maybe m) to be a member of the type class Eq, it is
-- required that the type m belongs to the type class Eq
instance Eq m ⇒ Eq (Maybe m) where
  Just x == Just y   = x == y
  Nothing == Nothing = True
  _ == _             = False
```

Just like it is possible to have higher- order functions, it is also possible to have higher-kinded types.

As far as the compiler is concerned, the only requirement for a type to be a member of a type class is that it should have a type correct instance declaration. There are of course additional semantic requirements that need to hold. These additional requirements are expressed as type class laws within the documentation of each type class.

Default implementations for some type classes can be generated automatically for data declarations using the deriving keyword:

```
data Bool = False | True
  deriving (Eq, Ord, Show, Read)
```

2.6.5. Newtype declarations

In cases where a datatype only has one constructor, a newtype declaration is used.

TODO:

2.6.6. Instances

2.6.6.1. Functor

Functors are types that can be used to **wrap and extract values** of other types, and allow us to **lift unary functions** over the wrapped value.

Definition

```
class Functor f where
  fmap :: (a → b) → f a → f b
  (<$) :: a → f b → f a
  {-# MINIMAL fmap #-}
```

Type Class Laws

The `fmap` function **should not change the structure** of the Functor, **only its elements**.

An instance of Functor has to abide by the following laws:

1) `fmap` has to preserve identity
   – `fmap id == id`
2) `fmap` has to preserve function composition
   – `fmap (g . h) == fmap g . fmap h`

fmap

```
-- <$> == fmap
(<$>) :: Functor f ⇒ (a → b) → f a → f b
```

Examples

```
(+1) <$> Nothing                          -- Nothing
(+1) <$> Just 1                           -- Just 2
```

Instances

```
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just x) = Just (f x)
```

2.6.6.2. Applicative

What if we want to lift an n-ary function?

Applicative Functors provide a **generic function to wrap values** (`pure`) and **generalized function application** (`<*>`).

Definition

```
class Functor f ⇒ Applicative f where
  pure :: a → f a
  (<*>) :: f (a → b) → f a → f b
  liftA2 :: (a → b → c) → f a → f b → f c
  (*>) :: f a → f b → f b
  (<*) :: f a → f b → f a
  {-# MINIMAL pure, ((<*>) | liftA2) #-}
```

Type Class Laws

1) Identity: `pure` has to preserve **identity**
   - `pure id <*> x == x`
2) Homomorphism: `pure` has to preserve **function application**
   - `pure (g x) == pure g <*> pure x`
3) Interchange: **Order of evaluation** must not matter when applying an effectful function to a pure argument
   - `x <*> pure y == pure (\f -> f y) <*> x`
4) Composition: `<*>` has to be **associative**
   - `x <*> (y <*> z) == (pure (.) <*> x <*> y) <*> z`

pure

We also need a function pure to lift 0-ary functions (constants):

```
pure :: a -> f a
f <$> x == (pure f) <*> x
```

Similarities to plain function application

```
($)   ::   (a -> b) ->   a ->   b
(<*>) :: f (a -> b) -> f a -> f b

(+) $ 11 $ 31                              -- 42
Just (+) <*> Just 11 <*> Just 31           -- Just 42
```

Examples

```
(+) <$> Nothing <*> Just 2                 -- Nothing
(+) <$> Just 1 <*> Just 2                  -- Just 3

(,) <$> [1,2] <*> [6,7]                    -- [(1,6),(1,7),(2,6),(2,7)]

-- intuition: all possible ways of multiplying the two input lists
pure (*) <*> [1,2] <*> [3,4]               -- [3,4,6,8]

-- in the context of lists, pure (*) == [(*)]
[(*),(+)] <*> [1,2] <*> [3,4]              -- [3,4,6,8,4,5,5,6]
```

Instances

```
instance Applicative Maybe where
  pure x = Just x
  Nothing <*> _ = Nothing
  (Just f) <*> mx = fmap f mx

instance Applicative IO where
  pure = return
  mf <*> mx = do
    f <- mf
    x <- mx
    return (f x)
```

2.6.6.3. Monad

So far, we have seen how functors and applicative functors help us to:

- apply pure functions to “effectful” arguments
- compose function application with multiple “effectful” arguments

What is missing is how to apply and compose “effectful” functions with “effectful” arguments.

Definition

```
class Applicative m ⇒ Monad m where
  (>>=) :: m a → (a → m b) → m b
  (>>) :: m a → m b → m b
  return :: a → m a
  {-# MINIMAL (>>=) #-}
```

Type Class Laws

1) Left identity
   - `return x >>= f = f x`
2) Right identity
   - `mx >>= return = mx`
3) Associativity
   - `(mx >>= f) >>= g = mx >>= (\x → (f x >>= g))`

Bind

Function that binds an effectful value into an effectful function

```
(>>=) :: m a → (a → m b) → m b
```

Kleisi arrow

The Kleisli arrow (<=<) is analogous to normal function composition, except that it works on monadic functions

```
(.)   ::              (b →   c) → (a →   b) → a →   c
(<=<) :: Monad m ⇒ (b → m c) → (a → m b) → a → m c
(f <=< g) x =  g x >>= f
```

Examples

```
safediv 8 0 >>= flip safediv 2              -- Nothing
safediv 8 2 >>= flip safediv 2              -- Just 2

safediv :: Int → Int → Maybe Int
safediv n m = if m == 0 then Nothing else Just $ div n m

do {x ← [1,2]; y ← [3,4]; return (x,y)}   -- [(1,3),(1,4),(2,3),(2,4)]
-- ⟺
[1,2] >>= \x → [3,4] >>= \y → return (x,y)
-- ⟺
[1,2] >>= \x → [3,4] >>= return . (x,)
-- ⟺
[1,2] >>= \x → (x,) <$> [3,4]
-- ⟺
(,) <$> [1,2] <*> [3,4]
```

Instances

```
instance Monad Maybe where
  Nothing >>= _ = Nothing
  (Just x) >>= f = f x

instance Monad [] where
  xs >>= f = [y | x <- xs, y <- f x]
```

2.6.6.4. Comparison

	Application operator	*Type of application operator*	*Function*	*Argument*	*Result*
Pure function application	juxtapose, `($)`	`(a -> b) -> a -> b`	pure	pure	pure
Functor	`fmap`, `(<$>)`	`(a -> b) -> f a -> f b`	pure	effectual	effectual
Applicative functor	`(<*>)`	`f (a -> b) -> f a -> f b`	pure	effectual	effectual
Monad	`(>>=)`	`m a -> (a -> m b) -> m b`	effectual	effectual	effectual

2.7. MODULES

- A collection of related values, types, classes, etc.
- Name: Identifier that starts with a capital letter.
- Naming convention for hierarchy: separated using "." (e.g. `Data.List`, `Data.List.NonEmpty`, `Data.Set`, ...)
- Each module `Dir.Name` must be contained in a file `Dir/Name.hs`.

```
module BinarySearchTree
( T, toList
) where

data T a = Leaf | Node (T a) a (T a)
  deriving Show

toList :: T a -> [a]

-- ...

import qualified BinarySearchTree
  as Set (T, toList)
```

2.8. LAZY EVALUATION

TODO:

2.9. EQUATIONAL REASONING

TODO:

2.10. SIDE EFFECTS

Interactive programs can be written in Haskell by using types to distinguish pure expressions from impure actions that may involve side effects.

```
IO a -- The type of actions that return a value of type a
```

One may also use a let statement within a do block to perform a pure computation. Note that the let statement here is different from the let … in … statement for expresisons.

```
act :: IO (Char, Char)
act = do {x ← getChar;
          y ← getChar;
          let {p = (x,y)};
          return p}
-- ⟺

act = do
        x ← getChar
        y ← getChar
        let p = (x,y)
        return p
```

Composing IO actions

```
(>>=) :: Monad m ⇒ m a → (a →  m b) →  m b

do {x ← getChar; putChar x}
-- ⟺
getChar >>= (\x → putChar x)
```

2.11. TODO:

- Effectful functions
- Dependent typing
- Mutable state + parallel programming
- denotative language / semantics (central passages)
- Referential transparency
  - **no** (mutable) variables
  - **no** assignments
  - **no** imperative control structures
  - all data structures are **immutable**
- Haskell generics

3. LAMBDA CALCULUS

– Alonzo Church

Term	***Definition***
free occurrence	

η conversion	a.k.a. ***η*** contraction denotes that for any ***f*** that does not contain any free occurances of ***x*** the following equality holds:
	`(\x → f x ) == f  -- TODO: rewrite in lambda notation`
	`f . g = \y → f (g x)`